Querying and Optimizing the Urban Data Platform

Group 11

1 Design Analytical Queries

1.1 Monthly taxi demand for each taxi zone.

Taxi demand is defined by the number of taxi rides requested by customers during a specific time period. Since we care about the monthly demand in this query, this can be translated to "the total number of taxi trips recorded, grouped by taxi zones and pickup month". For taxi zones, we group by the zone's name instead of ID, since many different zone IDs can point to the same zone (but different borough). To group by the month, we first create a pickup_month field during query runtime by transforming the pickup_date field.

1.2 Average trip distance under different weather conditions.

For each specific taxi trip, there are two weather condition values available. One for the weather at the pickup and one for the drop-off. We would like to know the trend over a specific weather condition, so instead of taking both, we considered only taking the pickup weather, since this is the condition people decide on when entering a cab. So the query should calculate the average distance of trips for a specific pickup weather condition. The weather conditions are considered by the `coco_code` field inside the weather dataset, combined later with a lookup table from: `https://dev.meteostat.net/formats.html`, and grouped by the weather field.

1.3 Relationship between air quality and taxi demand.

To investigate the relationship between air quality and taxi demand, taxi trips are grouped by their pickup hour. For each hour, demand is calculated as the total number of recorded trips, while the air-quality value is calculated as the average pickup air-quality measurement of the trips in that hour. Hours without an available air-quality measurement are excluded from the analysis. Finally, the Pearson correlation coefficient between hourly trip count and average air quality is computed. The resulting output contains the hourly demand and air-quality values, together with the overall correlation, making it possible to assess whether changes in air quality are correlated with taxi demand.

1.4 Taxi zones with the largest variation in demand under different weather conditions.

The variation under different weather conditions in a zone is implemented by selecting the difference between the busiest and least busy weather condition for a specific zone. However, to be able to compare the zones to each other we have to scale them since one zone might have more trips on average than others, and even though the scaled variance is higher in another. There are also cases where there is only 1 trip in a specific weather condition. These cases can be considered as noise so in order to measure a more meaningful variance, we excluded values where the score is below 0.01.

1.5 Peak travel hours for each day of the week.

To identify the peak travel hours, taxi trips are first converted from UTC to New York local time, ensuring that the results correspond to when passengers actually requested a ride. The trips are then grouped by day of the week and

pickup hour, where taxi demand is calculated as the total number of trips in each group. For every day of the week, the hourly groups are ranked by their trip count in descending order. The query returns the hour (or hours) with the highest demand for each weekday.

1.6 Monthly trends in taxi demand.

Similar to 1.1, to answer this query, a monthly *groupby* is required. This time, though, instead of also grouping by zone, this query is implemented to count the demand by summing the number of taxi trips, only by month. Again, just like in 1.1, we compute the *pickup_month* column during runtime.

2 Implement the Analytical Queries

The analytical queries were implemented using two strategies. Queries 2–5 were written in SQL and executed through PySpark, while Queries 1 and 6 use PySpark's DataFrame API. The framework was extended with an *analytics* layer that supports both SQL scripts and Python code as query definitions. These definitions are loaded and executed by a dedicated *runner* job.

3 Optimize Query Performance

We evaluated caching, partition pruning, and AQE for all six queries. Broadcast joins were evaluated only for Queries 1–3, as the remaining queries do not require joins. Each configuration was executed nine times, and median execution times and physical-plan comparisons are reported in the accompanying benchmark report.

Table 1: Optimization applicability by query.

Query	Broadcast join	Caching	Partition pruning	AQE
Query 1:	Taxi-zone lookup	Zone, date and also a monthly aggregation columns	February 2024 filter	Explained in report
Query 2:	Weather lookup	Weather and distance columns	February 2024 filter	Explained in report
Query 3:	Hourly air-quality relation	Pickup hour and air-quality columns	February 2024 filter	Explained in report
Query 4:	–	Zone and weather columns	February 2024 filter	Explained in report
Query 5:	–	Pickup timestamp	February 2024 filter	Explained in report
Query 6:	–	Date and monthly aggregation input	February 2024 filter	Explained in report

The following sections describe the optimization techniques evaluated for the analytical queries. Each technique was selected based on the structure of the query and the characteristics of the input data. Detailed execution times, and physical execution plans are presented in the accompanying benchmark report.

3.1 Broadcast join

Using a broadcast join, Spark sends a small relation to every executor so that it can be joined locally with a larger relation, avoiding a shuffle of

the larger dataset. This optimization was evaluated for Queries 1–3, which join taxi trips with a small taxi-zone lookup, weather-condition lookup, or hourly aggregated air-quality relation. Since these joins have already been performed in the integrated Gold table, the broadcast-join experiments use the corresponding Silver-layer tables.

3.2 Caching tables

Spark keeps a table or DataFrame in memory after computing it, allowing repeated queries to reuse it instead of reading it again from storage. Since the full Gold table does not fit in memory, each experiment caches a reduced view containing only the columns required by its query. This reduces repeated scans of the integrated taxi-trip dataset while preserving the query results. For queries 1 and 6 we also experimented with caching of intermediate aggregated table during execution.

3.3 Partition pruning

Delta tables can be partitioned by a column so that filters on that column allow Spark to skip irrelevant partitions. The integrated taxi-trip table was partitioned by year-month, and each query was evaluated using a February 2024 date filter. This reflects analytical workloads that request data for a specific time range and allows Spark to read only the relevant date partitions.

3.4 AQE

Adaptive Query Execution (AQE) adjusts Spark's physical execution plan using runtime statistics. It was evaluated for all queries because they involve operations such as aggregations, sorting, ranking, or joins that can require shuffles. AQE can coalesce small shuffle partitions and select more suitable execution strategies when the observed data size makes this beneficial. Detailed execution plans are available in the benchmark report.

4 Build Reusable Analytical Data Products

4.1 Overview and materialization approach

Four analytical data products are automatically generated from the integrated gold table by materializing the results of Queries 1-4: `monthly_zone_demand`, `average_trip_distance_by_weather`, `air_quality_vs_demand`, and `zone_weather_demand_variation`.

They are stored as separate Delta tables in a separate `analytics` directory, allowing analysts to query this precomputed results. Each refresh overwrites the product with the latest result and records metadata such as the source path, creation time, refresh time, and schema version.

4.2 Monthly zone demand

The `monthly_zone_demand` product contains the number of taxi trips for each pickup zone and month. It can be used by transport planners and analysts to identify changes in taxi demand between time and areas. Materializing this product avoids repeatedly grouping the full taxi-trip dataset by zone and month.

4.3 Average trip distance by weather condition

The `average_trip_distance_by_weather` product contains the average taxi-trip distance for each pickup weather condition. It can be used by analysts to examine how weather conditions relate to travel distance. Materializing this product avoids repeatedly aggregating trip distances by weather condition.

4.4 Air quality versus taxi demand

The `air_quality_vs_demand` product contains hourly taxi demand, average air-quality measurements, and their correlation. It can be used by analysts to examine whether changes in air quality are associated with taxi demand. Materializing this product avoids repeatedly calculating hourly aggregates and the correlation from the full dataset.

4.5 Zone weather demand variation

The `zone_weather_demand_variation` product measures how taxi demand varies across pickup weather conditions within each zone. The variation is calculated as the difference between the highest and lowest trip counts, divided by the average trip count across weather conditions. A higher value therefore indicates that demand in the zone differs more strongly between weather conditions.

It can be used by transport planners to identify zones with the largest weather-related differences in taxi demand. Materializing this product avoids repeatedly calculating weather-condition demand counts and their variation for each zone.

4.6 Storage, metadata, and refresh strategy

The above materialized tables are supporting efficient access and schema updates when needed.

During each refresh, the latest query result overwrites the previous product table. Every product row includes metadata describing the product name, source path, creation time, refresh time, refresh-run identifier, and schema version. A separate append-only Delta metadata table records the creation and refresh times of all products, preserving a history of refreshes.

The schemas, sample records, and refresh history of the products can be inspected in `notebooks/inspect_data_products.ipynb`.

5 Evaluate the Platform

5.1 Storage overhead of the analytical data products

By introducing analytical data products we are creating extra delta tables, which are in some cases taking up extra space compared to using only the `integrated_taxi_trips` table and querying directly from it. The spaces of each table represented in the table below. Results are measured in: `notebooks/storage_metrics/storage_overhead.ipynb`.

Table 2: Storage of delta tables

File	Delta
air_quality_vs_demand	38 972
average_trip_distance_by_weather	3 673
monthly_zone_demand	11 441
zone_weather_demand_variation	7 736
integrated_taxi_trips	325 797 243

These analytical products should contain the most up-to-date data, the contents are overwritten every execution, so there is no need to store previous versions. However delta was designed with the timetravel feature, meaning it does not actually delete old records, hence the size of these tables keeps increasing. We are able to vacuum old versions but this requires additional jobs and resources to schedule and execute.

Changes in the query execution plan, times and techniques are detailed throughout the benchmark report.